

# ICML 2023 | 数据科学代码生成，需要一把「可信」的尺子

港大、Meta AI 等提出 DS-1000：1000 道真实 StackOverflow 问题，执行式评测，还内建抵御记忆的防线

衡量代码生成模型时，我们习惯用 HumanEval、APPS 这类题目：算法谜题、面试风格、题面干净利落。它们考的是算法能力，却离大多数人真正的写代码日常有点远——尤其是数据科学。真实的数据科学任务往往不是「实现一个排序算法」，而是「我有这样一个 DataFrame，想给每一列加上它的倒数并按前缀命名，最 pythonic 的写法是什么」。

这类问题对模型的价值其实更高。数据科学离不开 NumPy、Pandas、PyTorch 这些专业库，光是记住它们庞杂的 API 就是不小的门槛，而代码生成模型恰好可以帮普通用户跨过这道门槛。问题是：社区一直缺一把能真正量出「进展」的尺子。多数已有基准要么是竞赛/面试式题目，要么用 BLEU 这类表层相似度来打分——随着模型越来越强，这类指标会越来越偏离程序员真正的意图，看着「像对的」文本反而拿高分，可代码根本跑不通。

这篇被 ICML 2023 接收的工作提出了 DS-1000：一个自然、可靠的数据科学代码生成基准。它从 StackOverflow 收集了 1000 道真实问题，覆盖七个常用 Python 库；用「执行代码」而非「比对文本」来判对错；并主动对题目做扰动，防止模型靠背答案蒙混过关。三件事合在一起，给了社区一把可信、且难以被刷分的标尺。

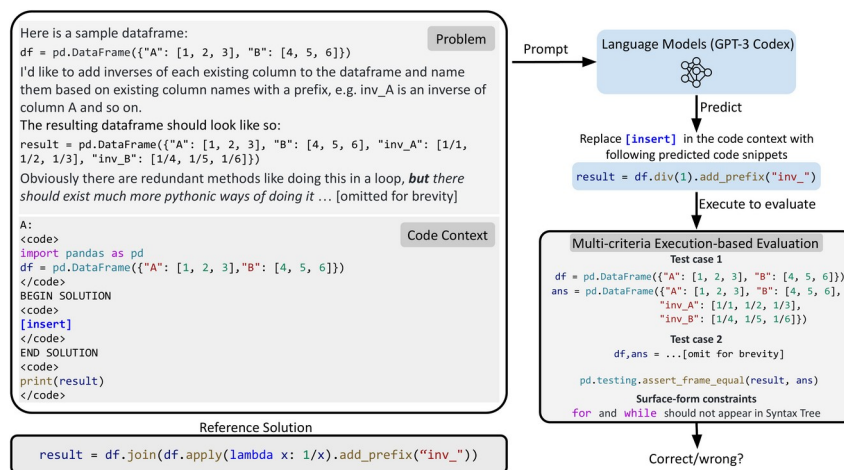


图 1 DS-1000 中的一个示例问题。模型需要把代码填入左侧提示里的 [insert] 处；补全后的程序会被执行，通过包含「测试用例」与「表层约束 (surface-form constraints)」的多准则评测，左下角给出一份参考解。

论文与代码：<https://arxiv.org/abs/2211.11501> | <https://github.com/HKUNLP/DS-1000> | 项目主页：<https://ds1000-code-gen.github.io/>

## 问题出在哪：题不真，尺不准

真实的数据科学提问，几乎从不长成教科书里那种干净的样子。在 StackOverflow 上，用户会带着一堆「上下文」发问：贴出自己写坏的代码、报出遇到的错误信息、给出「输入长这样、我想要的输出长那样」的具体例子。这些信息恰恰是模型该学会利用的，可过往基准往往把它们统统删掉，只留一句抽象的题面。

另一半问题出在评测本身。当模型还很弱时，用 BLEU 之类的表层指标凑合还看得过去；但模型一旦变强，「文本相似」和「真的能跑对」之间的裂缝会越拉越大——一段抄得很像参考答案、却逻辑错误的代码，照样能骗到高分。要衡量真实进展，就必须换成「把代码执行起来、用测试用例判对错」的可靠评测。DS-1000 正是冲着这两点去的：题目要自然、要带真实上下文，评测要靠执行、要可靠。

## 怎么造出来的：一条五阶段流水线

把 StackOverflow 上「野生」的问题，变成能自动评测又不怕被背答案的基准题，并不轻松。DS-1000 用一条五阶段流水线来完成这件事，每一道题、每一份解、每一个评测指标，都至少经过三位有经验的标注者交叉审核。

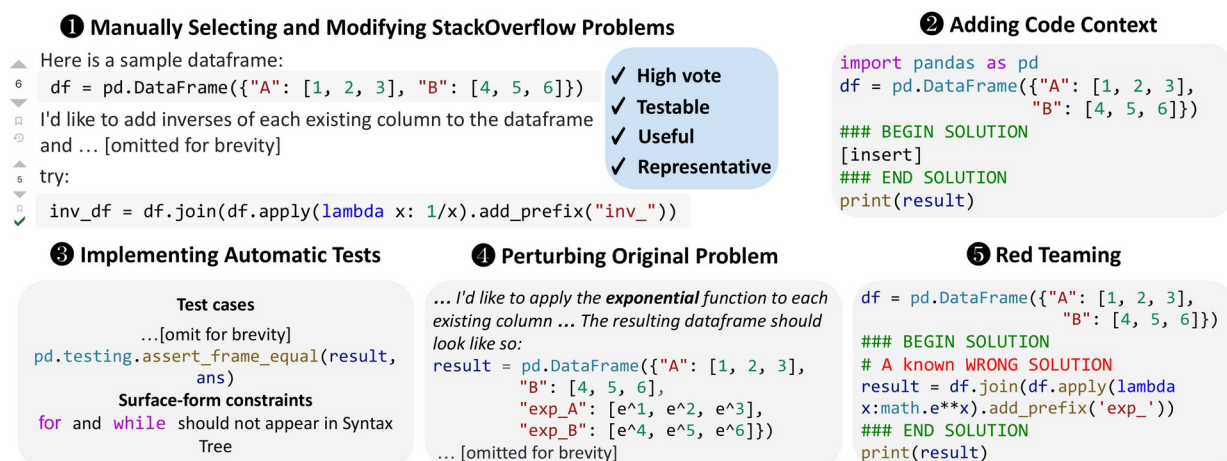


图 2 构建 DS-1000 的五阶段流程：(1) 人工挑选并改写高质量 StackOverflow 问题，(2) 补上带插入标记的代码上下文，(3) 实现自动测试（测试用例 + 表层约束），(4) 对原始问题做扰动以抵御记忆，(5) 对评测做红队测试。

第一步是挑题与改写：标注者按「高票、可测试、有用、有代表性」四条标准筛选帖子，并把口语化的原帖改写清楚。第二步补上代码上下文，用插入标记明确告诉模型「要在哪一处填代码」——也就是图 1 里的 [insert] 槽位。第三步实现自动测试：既跑测试用例检查功能是否正确，也在语法树上强制「表层约束」，比如禁止用 for / while 这类关键词或某些 API。前两步保证题目自然，第三步保证判分可靠。

真正的巧思在第四步——扰动。模型见过海量网页，很可能早就「背」下了 StackOverflow 的原答案。于是作者对每道原始问题做改写：「表层扰动」只改措辞、参考解不变；「语义扰

动」则改变题意、参考解随之改变；再加上一批刻意加难的「困难改写」。第五步是红队测试：评测脚本必须能挡住已知的错误解，才算合格。经过这一套，DS-1000 既贴近真实，又难以靠记忆蒙混。

## 数据长什么样：1000 道题，上下文更「厚」

DS-1000 最终包含 1000 道问题，来自 451 个不同的 StackOverflow 帖子，覆盖 NumPy、Pandas、TensorFlow、PyTorch、SciPy、Scikit-learn、Matplotlib 七个库。其中超过一半的题目是从原帖改造而来的，用来抵御记忆——具体是 152 道表层扰动、235 道语义扰动，以及 162 道困难改写。

从统计上看，平均每题带 1.6 个测试用例、约 140 个词的题面、3.6 行的参考解，还有 19.4% 的题目附带表层约束。140 词的平均长度远超同类数据科学数据集——DSP 平均 71.9 词、CoNaLa 只有 13.8 词。更长的题面意味着更丰富的真实上下文，这正是 DS-1000 想保留、而多数基准会丢掉的东西。

## 评测有多可靠：被接受的解，几乎都真的对

一个基准的可靠性，最终要落到「它判对的，是不是真的对」。DS-1000 用人工复核量化了这一点：在所有被判为正确的解里，只有 1.8% 其实是错的（样本级 false discovery rate）；被判为错误的解里，只有 0.5% 其实是对的（false omission rate）。换个更直观的角度，DS-1000 的问题级假阳性比例是 5.7%——作为对比，同一指标在 APPS 上曾高达 60%。执行式的多准则评测，把「误判」压到了很低的水平。

## 模型考得怎么样：最强也只有 43.3%

作者用 DS-1000 评测了来自三个家族的五个预训练代码模型。结果拉开了明显差距：表现最好的是 Insertion（插入 / 填空）格式下的 Codex-002，平均准确率也只有 43.3%——不算低，但离「解决」还很远，留出了大量提升空间；五个模型的成绩从 7.4% 一路铺到 43.3%。值得注意的是，Insertion 格式会额外提供右侧上下文，仅这一点就让 Codex-002 相比 Completion（补全）格式平均高出 4.1%，说明「填空式」的代码生成对数据科学任务确有价值。较弱的模型如 CodeGen-6B、InCoder-6B，在某些库上甚至跌破 5%。

## 为什么要扰动：一次揭穿记忆的探针

第四步的扰动到底有没有必要？作者用广为流传的 numpy-100 题集做了一次探针实验，结果很能说明问题。Codex-002 在原始 numpy-100 上能拿 72.5%，可一旦对题目做扰动，准确率直接塌到 40.6%；而且在 36% 的语义扰动情形里，模型依旧原封不动地吐出「原来那道题」的答案——此时那份答案已经是错的。这说明它是在回忆背下来的解，而不是在真正推理。

表 1 扰动前后的准确率对比（Codex-002）：numpy-100 这类在网上被反复传抄的题集，扰动后崩得厉害；而 DS-1000 因为更少被重复，扰动带来的下降要温和得多。

| 设置        | 原始准确率 | 扰动后   | 表层扰动降幅 | 语义扰动降幅 |
|-----------|-------|-------|--------|--------|
| numpy-100 | 72.5% | 40.6% | -50.8% | -23.6% |
| DS-1000   | —     | —     | -3.4%  | -9.0%  |

对照之下，同样的扰动放到 DS-1000 上，准确率的下降要温和得多（表层约 3.4%、语义约 9.0%）——因为这些题目在网上被重复的次数本就更少，记忆的空间更小。这从两头印证了一件事：记忆是真实存在的风险，而主动扰动是一种简单有效的防御，能让基准对未来更强的模型也保持挑战性。

### 小结：一把真实、可信、抗记忆的尺子

DS-1000 给数据科学代码生成提供了一把难得的标尺：题目取自真实的 StackOverflow 提问，因而贴近实际；评测靠执行代码、并叠加表层约束，因而可信、误判极低；再用刻意扰动挡住记忆，因而难以被刷分、也对未来的模型持续有效。而「即便最强模型也只有 43.3%」这个数字本身，就清楚地标出了这个方向还有多大的空间待填。对做数据科学代码生成的研究者来说，它既是一个诚实的测量工具，也是一份还远未完成的任务清单。